

GENETIC ALGORITHM FOR KNAPSACK PROBLEM

-SRIKANTH RAVI JOIS (230002071) & J RAMULU NAIK
(230002031)

CONTENTS

- ABSTRACT
- IMPLEMENTATION
- CONCLUSION
- RECOMMENDATIONS TO IMPROVE THE SOLUTION
- CONTRIBUTION
- REFERENCES

ABSTRACT

THE KNAPSACK PROBLEM

There are different variants of the knapsack problem like the Fractional Knapsack Problem, 0/1 Knapsack Problem, Bounded Knapsack Problem and the Unbounded knapsack Problem. This project attempts to solve the 0/1 Knapsack Problem using the Genetic Algorithm.

In 0/1 Knapsack Problem, we are given N items where each item has some weight (w_i) and value (v_i) associated with it. We are also given a bag with capacity W . The target is to put the items into the bag such that the sum of values associated with them is the maximum possible. It is to be noted that an item can either be selected or not (fractional solutions are not allowed).

GENETIC ALGORITHM

A genetic algorithm (GA) is a computational technique that uses principles of evolution and genetics to find solutions to problems. GAs are based on the idea of 'Survival of the Fittest' which states that the organisms which are most suited to their environment are more likely to survive and reproduce.

Here are some key concepts of genetic algorithms:

- **Representation:** Each solution is represented as an individual called a chromosome.
- **Fitness:** Each chromosome has a fitness value that's related to the problem's cost function.
- **Selection:** The algorithm selects the best solutions to be parents for the next generation.
- **Crossover:** The algorithm creates new solutions by exchanging a random segment between two parents.
- **Mutation:** The algorithm introduces random points to avoid local minima.
- **Termination:** The algorithm ends when the desired solution is reached or the maximum number of iterations is reached

IMPLEMENTATION

In 0/1 knapsack problem there are multiple items each with a certain weight and value. The objective is to fit items into a knapsack of certain weight threshold such that value of items is maximum.

Representation:

While implementing GA for knapsack problem we define each chromosome as an array of bits where each bit represents whether the item is present in the knapsack or not. The algorithm initially generates a set of chromosomes randomly. The set of chromosomes is called a population.

Fitness:

In the context of the knapsack problem the fitness value is defined as the total value of all the items present in the knapsack. The chromosomes whose weights exceed the maximum capacity of the knapsack are given a fitness value of 0. The algorithm aims to maximize the fitness value over several generations.

Selection:

The selection process involves choosing of chromosomes from the population for crossing to produce the next generations.

Two of the commonly used selection methods are Tournament Selection and Roulette Wheel.

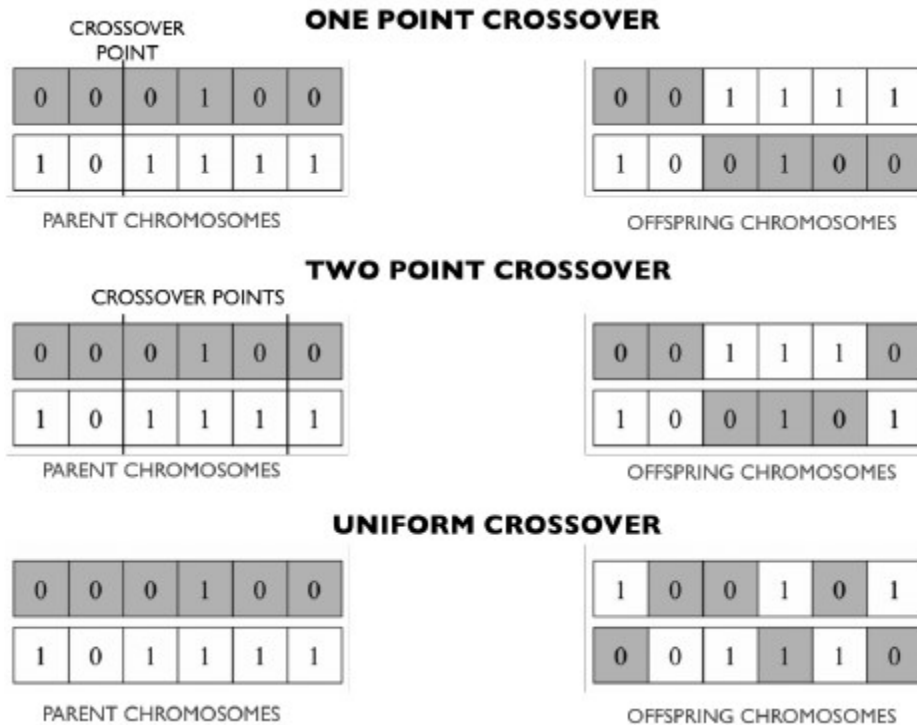
In Tournament Selection method the parents are chosen at random and made to undergo crossing.

In Roulette Wheel method the chromosomes with higher fitness values are preferred of those with lower fitness values. As a result, chromosomes exceeding maximum capacity will be neglected.

Crossover:

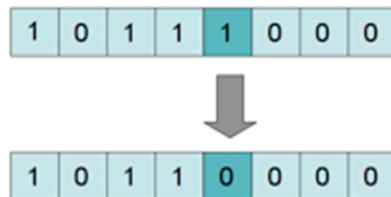
In Crossover the parent chromosomes selected in the previous step exchange some of their values to produce two offsprings. These offsprings constitute the next generation.

There are three types of Crossovers – Single Point Crossover, Multi Point Crossover and Uniform Crossover.



Mutation:

After crossing, there is a small chance that the offspring may mutate at a point resulting in a mutated offspring. The mutation rate for a genetic algorithm is often low (around 0.01 to 0.02). This may be beneficial or detrimental to the overall fitness of the chromosome as it may increase (0 to 1 mutation) or decrease (1 to 0 mutation) the chromosome's fitness.



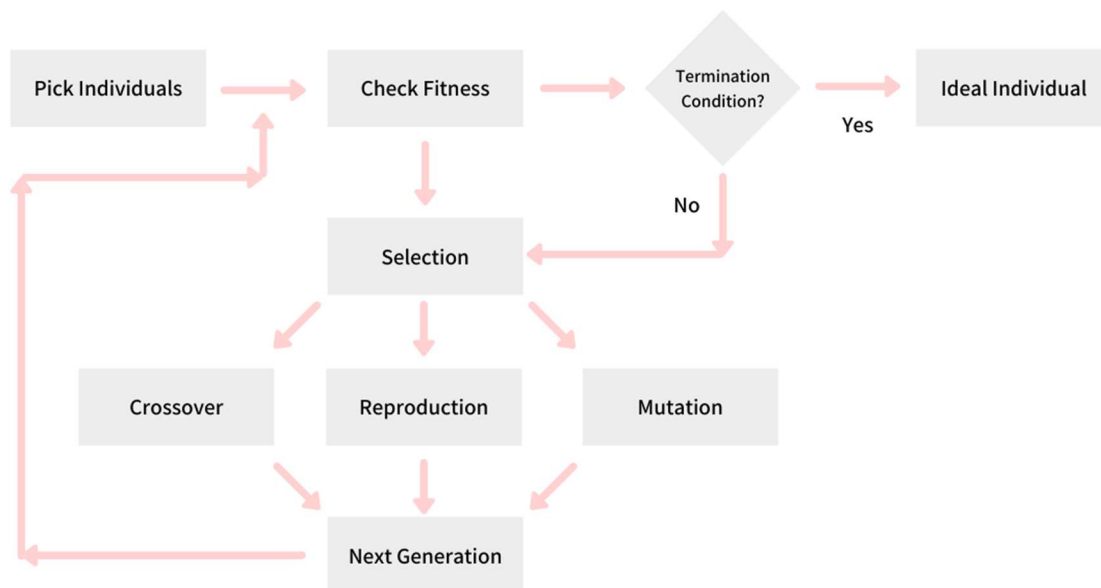
Termination:

The algorithm repeats with every generation until it reaches a termination point. The point may be a limit set on the number of generations, a range around the maximum possible fitness value or a time limit.

CONCLUSION:

This report gives a description on how to use Genetic Algorithm to solve Knapsack Problem using Tournament Selection method. In the initial stage of the algorithm, we

initialize a set of chromosomes each representing a solution. The total value of each solution is calculated and assigned as the fitness value of the chromosome. If the total weight of the items exceeds the maximum capacity of the knapsack, we assign the chromosome a fitness value of 0. In the next step chromosomes are selected randomly as parents which undergo crossing to produce the offsprings. There is a small chance of the offsprings mutating to cause changes in the offspring. This process continues over several generations until the limit on the number of generations is reached at which point the algorithm terminates.



RECOMMENDATIONS TO IMPROVE SOLUTION:

- Increase population size: More diverse genetic pool.
- Adjust genetic operators: Modify crossover and mutation rates.
- Elite preservation: Safeguard the best chromosomes.
- Elitism strategy: Preserve the top performing chromosomes.
- Fine-tune parameter settings: Experiment with population size, crossover rate, mutation rate, and elitism.
- Hybrid approaches: Combine genetic algorithms with local search or other optimization techniques to refine solutions.
- Adaptive parameter control: Dynamically adjust genetic operator parameters during evolution based on population diversity and fitness trends.

CONTRIBUTION:

- Report: Srikanth Ravi Jois
- Power Point: J Ramulu Naik
- Code: Srikanth Ravi Jois & J Ramulu Naik

REFERENCES:

<https://arpitbhayani.me/blogs/genetic-knapsack/>

<https://www.geeksforgeeks.org/0-1-knapsack-problem-dp-10/>

<https://github.com/asadelkhosh/Genetic-Algorithm-For-Knapsack-Problem>

<https://github.com/edmilsonrobson/0-1-Knapsack-Problem-with-Genetic-Algorithms/tree/master>